Studio Nx

P1-09 비밀번호 변경 · 탈퇴 — 함수 설명 합본 (44~49)

2026-10-12

목차

1. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/44_user_service_check_current_password.md
2. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/45_user_service_change_password.md
3. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/46_user_service_withdraw.md
4. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/47_users_route_change_password.md
5. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/48_users_route_withdraw.md
6. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/49_retention_purge_withdrawn_accounts.md

Backend(FastAPI/Python) 44 — `check_current_password()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-09 비밀번호 변경·탈퇴: 비밀번호 재확인 (PRD 01 FR-107, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	43 `GET /profile-images/{name}` · 45 `change_password()`

1. 하는 일

이미 로그인한 회원에게 비밀번호를 한 번 더 확인합니다. 맞으면 아무것도 하지 않고(`None`), 틀리면 400으로 응답합니다.

저장된 비밀번호 `Abcdef1!`에 넣은 값	결과
`Abcdef1!`	통과합니다 (`None`)
`Abcdef1?`, `abcdef1!`, `" Abcdef1!"`, 빈 문자열	400 `WRONG_PASSWORD` "Password is incorrect."
100자 (bcrypt 한도인 72바이트를 넘음)	똑같이 400입니다 (500이 아닙니다)
해시가 `"!"`인 경우 (보관 기간 정리를 거친 계정)	어떤 값이든 400입니다

2. 언제 쓰나

쓰는 곳	다시 묻는 이유
45번 `change_password`	"지금 비밀번호" 칸입니다. 잠깐 자리를 비운 사이에 다른 사람이 비밀번호를 바꾸지 못하게 합니다
46번 `withdraw`	탈퇴 창의 비밀번호 칸입니다 (FIX-01 `promptDialog`). 되돌릴 수 없는 일이기 때문입니다

로그인(29번 `authenticate`)과 다른 점

	29번 로그인	44번 재확인
회원	아직 모릅니다 (이메일로 찾습니다)	이미 압니다 (토큰의 회원)
틀렸을 때	401 `INVALID_CREDENTIALS`	**400** `WRONG_PASSWORD`
응답 시간 맞추기 (가짜 해시)	합니다. 없는 이메일인지 숨기기 위해서입니다	필요 없습니다. 숨길 것이 없기 때문입니다

3. 코드

import (파일 맨 위)

```
from app.core.errors import AppError, ErrorCode
from app.core.security import verify_password
```

`verify_password` **줄을 이번에 추가했습니다** (나머지 import는 P1-08 그대로입니다).

함수

```
def check_current_password(user: User, password: str) -> None:
    if not verify_password(password, user.password_hash):
        raise AppError(ErrorCode.WRONG_PASSWORD, 400, "Password is incorrect.")
```

문법 풀이

import 문법	뜻
`from app.core.security import verify_password`	P1-04(16)에서 만든 함수입니다. bcrypt로 비교하고, 오류가 나면 `False`를 돌려줍니다

식	뜻
`-> None`	돌려주는 값이 없습니다. "통과하면 조용히 끝나고, 아니면 오류를 낸다"는 검사 함수입니다
`if not …: raise …`	틀렸을 때만 멈춥니다

4. 흐름

1. 넣은 비밀번호를 저장된 해시와 비교합니다.
2. 틀리면 400으로 응답하고, 맞으면 그대로 끝납니다.

5. 왜 이렇게

선택	이유
401이 아니라 **400**으로 응답	화면의 `api.js`(P1-02 02)는 401을 받으면 "토큰이 만료되었다"고 판단해서 재발급을 시도하고, 그래도 401이면 로그인 화면으로 보냅니다. 비밀번호를 잘못 입력했을 뿐인데 **로그아웃되면 안 됩니다**
별도 코드 `WRONG_PASSWORD`	화면이 "Current password is incorrect."처럼 그 칸에 문구를 붙일 수 있게 합니다 (P1-10)
두 줄이지만 함수로 분리	45번과 46번이 같은 규칙을 씁니다. 문구와 코드가 한곳에 모입니다
DB를 다시 읽지 않음	06번 `CurrentUser`가 이미 최신 회원 정보를 읽어 왔기 때문입니다

알아 둘 점 (여러 번 틀리는 경우): 로그인한 사람이 이 창에서 비밀번호를 계속 바꿔 입력하며 맞혀 볼 수는 있습니다. 하지만 이미 그 계정으로 로그인한 상태라서 얻을 것이 적습니다. 횟수 제한은 로그인 반복 제한과 함께 나중에 검토합니다.

6. 시험 (`tests/test_user_service.py`에 7개를 추가해서 모두 63개)

시험	확인한 것
맞는 비밀번호	`None`을 돌려줍니다
틀린 경우 5가지	한 글자 다름, 대소문자 다름, 앞에 공백, 빈 문자열, 100자는 모두 400 `WRONG_PASSWORD` "Password is incorrect."입니다
해시 `"!"`	400입니다

전체 시험 321개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 45 — `change_password()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-09 비밀번호 변경·탈퇴: 비밀번호 변경 (PRD 01 FR-107)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	44 `check_current_password()` · 46 `withdraw()`

1. 하는 일

지금 비밀번호를 확인한 다음, 새 비밀번호가 규칙에 맞으면 새 해시로 바꿔 저장합니다.

지금 비밀번호가 `Abcdef1!` 일 때 보낸 값 (지금 / 새)	결과
`Abcdef1!` / `Newpass9#`	저장합니다. 이후로는 `Newpass9#`로만 로그인할 수 있습니다
`Abcdef1?` / `Newpass9#`	400 `WRONG_PASSWORD` "Password is incorrect." (44)
`Abcdef1!` / `abcdefgh`	422 "Check the password requirements."와 `details: {"new_password": ["upper", "number", "special"]}`
`Abcdef1!` / `Ab1!`	422와 `["length"]`
`Abcdef1!` / `Abcdef1!` 한글	422와 `["allowed"]`
`Abcdef1!` / `Abcdef1!` (같음)	422 **"Use a different password."**와 `details: {"new_password": "same_as_current"}`
다른 회원이 `Newpass9#`를 쓰고 있음	상관없이 저장합니다

오류가 나면 DB의 해시는 그대로 남습니다.

2. 언제 쓰나

쓰는 곳	
`PUT /users/me/password` (47)	userInfo.html의 비밀번호 칸 3개(지금, 새, 확인)를 채우고 Change password를 누를 때 씁니다 (P1-10)

사용자 결정 (2026-10-07)

새 비밀번호가 지금 비밀번호와 같으면 막습니다. 템플릿에는 "(선택)"으로 적혀 있었습니다.

- 이것은 **같은 회원 안에서** 비교하는 것입니다. 다른 회원과 비밀번호가 겹치는지는 검사하지 않습니다. 겹쳐도 문제가 없고, 오히려 검사하면 "누군가 이 비밀번호를 쓰고 있다"는 단서가 되기 때문입니다. 또 해시마다 소금(salt)이 달라서 비교할 수도 없습니다.
- 예전에 쓰던 비밀번호들까지 막는 기능(비밀번호 이력)은 범위 밖입니다.

3. 코드

import (파일 맨 위)

```
from app.core.security import check_password_rules, hash_password, verify_password
```

`check_password_rules`와 `hash_password`를 이번에 추가했습니다 (44번에서 추가한 `verify_password` 줄에 이어서).

함수

```
def change_password(db: Session, user: User, current_password: str, new_password: str) -> None:
    check_current_password(user, current_password)
    failed = check_password_rules(new_password)
    if failed:
        raise AppError(
            ErrorCode.VALIDATION_ERROR,
            422,
            "Check the password requirements.",
            details={"new_password": failed},
        )
    if verify_password(new_password, user.password_hash):
        raise AppError(
            ErrorCode.VALIDATION_ERROR,
            422,
            "Use a different password.",
            details={"new_password": "same_as_current"},
        )
    user.password_hash = hash_password(new_password)
    db.commit()
```

문법 풀이

import 문법	뜻
`check_password_rules`, `hash_password`	P1-04(14, 15)에서 만든 함수입니다. 가입과 같은 규칙으로 검사하고 해시를 만듭니다

식	뜻
`if failed:`	빈 목록이면 거짓이고, 실패한 조건이 하나라도 있으면 참입니다
`verify_password(new_password, user.password_hash)`	새 비밀번호를 **지금 해시**에 대어 봅니다. 맞으면 지금 비밀번호와 같다는 뜻입니다
`-> None`	돌려줄 값이 없습니다. 라우터는 204(본문 없음)로 응답합니다

4. 흐름

1. 지금 비밀번호를 확인하고, 틀리면 400으로 응답합니다.
2. 새 비밀번호의 규칙을 검사하고, 틀리면 422와 실패한 조건을 돌려줍니다.
3. 지금 비밀번호와 같으면 422로 응답합니다.
4. 새 해시로 저장합니다.

5. 왜 이렇게

선택	이유
지금 비밀번호를 **먼저** 확인	다른 사람이 내 자리에서 새 비밀번호 규칙을 시험해 보는 것조차 못 하게, 본인 확인을 가장 먼저 합니다
규칙 검사를 "같음" 검사보다 먼저	지금 비밀번호는 이미 규칙을 통과했던 값이라서 순서가 결과를 바꾸지 않습니다. 그래서 DB를 보지 않는 가벼운 검사를 앞에 둡니다 (P1-04와 같은 원칙)
"같음" 비교에 `verify_password` 사용	서버에는 지금 비밀번호의 원문이 없으므로, 해시에 대어 봐야 알 수 있습니다. 지금 비밀번호 칸의 값과 글자로 비교(`current == new`)할 수도 있지만, 해시와 비교해야 "실제로 저장된 것과 같다"는 것이 확실해집니다
`details`의 칸 이름을 `new_password`로 함	화면(P1-10)이 그 칸 아래에 문구를 붙일 수 있게 했습니다. 요청 본문의 칸 이름과 같습니다
`refresh`를 하지 않음	돌려줄 정보가 없기 때문입니다 (응답 204)

알아 둘 점 (다른 기기): 비밀번호를 바꿔도 다른 브라우저에 이미 있는 리프레시 쿠키(최대 3일)는 그대로 동작합니다. 리프레시 토큰을 DB에 저장하지 않는 설계이기 때문입니다 (P1-05 34번 §5와 같은 한계). 필요해지면 "이 시각 이전에 만든 토큰은 무효"라는 값을 회원 행에 두는 방법이 있습니다 (토큰의 `iat`, P1-05 26).

6. 시험 (`tests/test_user_service.py`에 7개를 추가해서 모두 70개, 실제 MySQL로 실행)

시험	확인한 것
바꾸기	DB 해시가 바뀌고, 새 비밀번호는 맞고 옛 비밀번호는 틀립니다
지금 비밀번호가 틀림	400 `WRONG_PASSWORD`이고, 해시는 그대로입니다
규칙 3가지	약한 비밀번호, 짧은 비밀번호, 한글과 공백은 422이고, `details.new_password`에 실패 목록이 담깁니다
지금과 같음	422 "Use a different password."와 `same_as_current`이고, 해시는 그대로입니다
다른 회원과 같은 비밀번호	저장됩니다

전체 시험 328개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 46 — `withdraw()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-09 비밀번호 변경·탈퇴: 회원 탈퇴(소프트 삭제) (PRD 01 FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	45 `change_password()` · 47 `PUT /users/me/password`

1. 하는 일

비밀번호를 확인한 뒤 회원을 **탈퇴 상태**로 바꿉니다. 회원 행은 지우지 않으며(소프트 삭제), 프로필 사진은 지웁니다.

바뀌는 것	바뀌지 않는 것
`status` → `DELETED`	이메일과 닉네임 (3일 동안 재가입을 막는 데 씁니다)
`deleted_at` → 지금 시각 (UTC)	회원 행 자체
`profile_image_path` → `None`, 사진 파일 삭제	내가 OWNER인 프로젝트

상황	결과
비밀번호가 맞음	위 표처럼 바뀝니다
비밀번호가 틀림	400 `WRONG_PASSWORD`이고, 아무것도 바뀌지 않으며 사진도 그대로입니다
저장 실패	오류를 그대로 전달하고, 사진 파일도 남습니다

탈퇴한 뒤에 일어나는 일 (다른 함수들이 이미 처리합니다)

무엇	결과	막는 곳
로그인	401 (이유는 숨깁니다)	29 `authenticate`
남아 있는 Access Token (최대 30분)	401	06 `get_current_user`
자동 로그인 쿠키로 재발급	401	30 `get_active_user`
3일 안에 같은 이메일이나 닉네임으로 가입	409 `REJOIN_TOO_SOON`	19, 20 (P1-04)
사진 주소	404	40
3일 뒤	이메일, 닉네임과 남은 개인 정보를 비웁니다	49번 보관 기간 정리

2. 언제 쓰나

쓰는 곳	
`DELETE /users/me` (48)	본문 `{"password": "…"}`를 받아 이 함수를 부르고, 리프레시 쿠키를 지운 뒤 204로 응답합니다
화면	userInfo.html의 Delete account → FIX-01 `promptDialog` (비밀번호만 받고 체크박스는 없음, P1-10)

3. 코드

import (파일 맨 위)

```
import re
from datetime import UTC, datetime
from pathlib import Path, PurePosixPath
…
```

`from datetime import UTC, datetime`을 이번에 추가했습니다. `UserStatus`는 40번에서 이미 넣었습니다.

함수

```
def withdraw(db: Session, user: User, password: str) -> None:
    check_current_password(user, password)
    user.status = UserStatus.DELETED
    user.deleted_at = datetime.now(UTC).replace(tzinfo=None)
    delete_profile_image(db, user)
```

문법 풀이

import 문법	뜻
`from datetime import UTC, datetime`	지금 시각(UTC 기준)을 구할 때 씁니다

식	뜻
`datetime.now(UTC).replace(tzinfo=None)`	지금 UTC 시각에서 시간대 표시를 떼어 냅니다. DB의 `DATETIME` 칸은 시간대를 저장하지 않고 항상 UTC로 쓰기 때문입니다 (P1-04와 같음)
`user.status = …`, `user.deleted_at = …`	값만 바꿉니다. 아직 DB에 저장되지 않았습니다
`delete_profile_image(db, user)`	39번 함수입니다. 사진 경로를 비우고 **`commit`**하므로, 위의 두 줄도 이 `commit`으로 함께 저장됩니다. 저장한 뒤 파일을 지웁니다

4. 흐름

1. 비밀번호를 확인하고, 틀리면 400으로 응답합니다 (아무것도 바뀌지 않습니다).
2. 상태와 탈퇴 시각을 바꿉니다 (아직 저장하기 전입니다).
3. 39번을 부릅니다. 사진 경로를 비우고 한 번에 저장한 뒤, 사진 파일을 지웁니다.

5. 왜 이렇게

선택	이유
39번을 **마지막에** 불러서 저장을 한 번으로 처리	템플릿이 "commit은 마지막에 한 번"을 요구했습니다. 상태만 저장되고 사진은 남는(또는 그 반대인) 반쪽짜리 탈퇴가 생기지 않습니다. 39번을 만들 때 이렇게 쓰일 것을 미리 정해 두었습니다 (39 §5)
이메일과 닉네임을 바로 비우지 않음	2026-10-03 결정입니다. 탈퇴하고 같은 이메일로 곧바로 다시 가입해서 제재나 기록을 피하는 것을 3일 동안 막습니다
프로젝트를 남김	2026-10-03 결정입니다. 함께 작업한 사람의 곡이 사라지면 안 되기 때문입니다. 다른 사람에게는 'Deleted user'로 보입니다 (Phase 6, 추적 방지 ①)

선택	이유
리프레시 쿠키는 라우터에서 지움	쿠키는 **응답**에 붙는 것이라서, DB를 다루는 서비스와 나눴습니다. 쿠키를 지우지 못하더라도 30번이 재발급을 막습니다
사진 파일은 바로 지움	재가입을 막는 데 쓰이지 않으므로 3일을 기다릴 이유가 없습니다

6. 시험 (`tests/test_user_service.py`에 4개를 추가해서 모두 74개, 실제 MySQL과 임시 폴더로 실행)

시험	확인한 것
탈퇴	DB에 `DELETED`와 지금 시각이 저장되고, 이메일과 닉네임은 그대로이며, 사진 경로는 `None`이 되고 파일이 지워지며, OWNER 프로젝트는 그대로입니다
비밀번호가 틀림	400이고, 상태는 `ACTIVE`이며 사진 파일도 그대로입니다
탈퇴한 뒤	로그인 401, 재발급 401, 같은 이메일과 닉네임은 `REJOIN_TOO_SOON`입니다
저장 실패	오류가 그대로 전달되고, 사진 파일이 남습니다

전체 시험 332개가 통과했고, `ruff` 검사도 통과했습니다. (P1-08 사이트 시험이 남긴 빈 `storage/profiles/` 폴더 내용을 지워서, 이제 `.gitkeep`만 남아 있습니다.)

Backend(FastAPI/Python) 47 — `PUT /api/v1/users/me/password`

항목	내용
파일	`studionx/backend/app/api/v1/users.py`의 `change_password()`
작업	P1-09 비밀번호 변경·탈퇴: 비밀번호 변경 (PRD 01 FR-107)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	46 `withdraw()` · 48 `DELETE /users/me`

1. 하는 일

요청 본문 (지금 비밀번호 `Abcdef1!`)	응답
`{"current_password": "Abcdef1!", "new_password": "Newpass9#"}`	**204**(본문 없음)이고, 이후로는 `Newpass9#`로만 로그인할 수 있습니다
지금 비밀번호가 틀림	**400** `WRONG_PASSWORD` "Password is incorrect."
새 비밀번호가 `abcdefgh`	422 "Check the password requirements."와 `details: {"new_password": ["upper", "number", "special"]}`
새 비밀번호가 지금과 같음	422 "Use a different password."와 `details: {"new_password": "same_as_current"}`
`current_password`가 빈 문자열	422 (스키마)이고, `details`에 `current_password`가 담깁니다
토큰 없음	401

2. 언제 쓰나

화면 (userInfo.html, P1-10)	
Current password, New password, Confirm을 채우고 Change password	`api.put('/users/me/password', { current_password, new_password })`로 보냅니다. 확인 칸과의 비교는 화면이 합니다 (가입 16번과 같은 방식)
400 `WRONG_PASSWORD`	지금 비밀번호 칸 아래에 "Current password is incorrect."를 표시합니다
422 `details.new_password`	새 비밀번호 칸 아래에 문구를 표시합니다

3. 코드

import (파일 맨 위: 41번과 같고, 새로 추가한 것은 없습니다)

함수 (경로와 응답 설정은 템플릿 그대로입니다)

```
@router.put(
    "/me/password",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={400: {"model": ErrorResponse}, 422: {"model": ErrorResponse}},
)
def change_password(data: PasswordChangeRequest, user: CurrentUser, db: Db) -> None:
    user_service.change_password(db, user, data.current_password, data.new_password)
```

문법 풀이

식	뜻
`@router.put(…)`	**PUT**은 "이 값으로 바꿔 놓으라"는 요청입니다. 비밀번호는 통째로 새 값이 되므로 PATCH(일부 수정)가 아니라 PUT을 씁니다
`status_code=status.HTTP_204_NO_CONTENT`	성공하면 204로 응답합니다. 돌려줄 내용이 없기 때문입니다
`data: PasswordChangeRequest`	템플릿 스키마입니다. 두 칸 모두 1~100자여야 하며, 빈 칸이면 422입니다
`-> None`	본문이 없습니다

4. 흐름

토큰을 확인하고(06), 본문을 검사한 뒤(스키마), 45번(지금 비밀번호 확인 → 규칙 검사 → 같음 검사 → 저장)을 실행하고 204로 응답합니다.

5. 왜 이렇게

선택	이유
라우터는 한 줄	규칙은 44번과 45번에서 이미 시험을 마쳤습니다
204로 응답하고 새 토큰을 주지 않음	Access Token에는 회원 번호만 담겨 있어서, 비밀번호가 바뀌어도 그대로 쓸 수 있습니다. 그래서 다시 로그인시키지 않습니다
틀리면 400	44번 §5에서 설명했듯이, 401로 응답하면 화면이 로그아웃시킵니다
새 비밀번호 확인 칸은 서버로 보내지 않음	두 칸이 같은지는 화면이 확인합니다 (가입과 같음). 서버는 같은 값을 두 번 받아도 할 일이 없습니다

6. 시험 (`tests/test_user_routes.py`에 6개를 추가해서 모두 32개, 실제 앱과 MySQL로 실행)

시험	확인한 것
바꾸기	204와 본문 없음을 확인한 뒤, **실제 로그인 API**로 새 비밀번호는 200, 옛 비밀번호는 401임을 확인했습니다
지금 비밀번호가 틀림	400 `WRONG_PASSWORD` "Password is incorrect."
규칙 위반, 지금과 같음	422와 문구, `details.new_password`
지금 비밀번호가 빈 칸	422이고, `details`에 `current_password`가 담깁니다
토큰 없음	401

전체 시험 338개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 48 — `DELETE /api/v1/users/me`

항목	내용
파일	`studionx/backend/app/api/v1/users.py`의 `withdraw()`
작업	P1-09 비밀번호 변경·탈퇴: 회원 탈퇴 (PRD 01 FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	47 `PUT /users/me/password` · 49 `purge_withdrawn_accounts()`

1. 하는 일

요청	응답
본문 `{"password": "Abcdef1!"}` (맞음)	**204**와 `Set-Cookie: snx_refresh=""; Max-Age=0; Path=/api/v1/auth; HttpOnly; SameSite=lax`(쿠키 삭제)
비밀번호가 틀림	400 `WRONG_PASSWORD`이고, 쿠키는 그대로라서 계속 로그인 상태입니다
본문 없음, `{}`, 빈 문자열	422이고, 탈퇴하지 않습니다
토큰 없음	401

204로 응답한 뒤 (실제 앱으로 확인했습니다)

무엇	결과
남아 있는 Access Token으로 `GET /users/me`	401
`POST /auth/refresh`	401입니다. 쿠키가 지워졌기 때문입니다
같은 이메일과 비밀번호로 로그인	401
사진 주소	404

2. 언제 쓰나

화면 (userInfo.html, P1-10)	
Delete account → FIX-01 `promptDialog` (비밀번호 입력, `check` 안에서 이 API를 부름)	`api.delete('/users/me', { body: { password } })`
204	`clearAccessToken()`을 부르고 `home.html`로 이동합니다
400 `WRONG_PASSWORD`	창 안에 "Password is incorrect."를 표시하고, 창은 닫히지 않습니다 (21번 `check`)

함께 고친 것 (화면 쪽 메모)

`api.js`(P1-02)의 `api.delete(path, options)`는 두 번째 인자가 **옵션**입니다. 그래서 본문은 `{ body: … }` 안에 넣어야 합니다. 그런데 `userinfo.js` 템플릿 주석과 FIX-01 21번 문서의 예시가 `api.delete('/users/me', { password })`로 되어 있었습니다. 이대로 쓰면 본문 없이 요청이 가서 422가 납니다. 그래서 두 곳 모두 `{ body: { password } }`로 고쳤습니다.

3. 코드

import (파일 맨 위)

```
from app.api.deps import CurrentUser, Db
from app.api.v1.auth import clear_refresh_cookie
```

`clear_refresh_cookie` 줄을 이번에 추가했습니다 (P1-05 31).

함수 (경로와 응답 설정은 템플릿 그대로입니다)

```
@router.delete(
    "/me",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={400: {"model": ErrorResponse}},
)
def withdraw(data: WithdrawRequest, response: Response, user: CurrentUser, db: Db) -> None:
    user_service.withdraw(db, user, data.password)
    clear_refresh_cookie(response)
```

문법 풀이

import 문법	뜻
`from app.api.v1.auth import clear_refresh_cookie`	로그아웃(34)이 쓰는 쿠키 삭제 함수입니다. 쿠키를 심을 때와 같은 속성으로 지워야 브라우저가 같은 쿠키로 알아봅니다

식	뜻
`@router.delete("/me", …)`	`DELETE /api/v1/users/me`입니다. 같은 주소의 `GET`, `PATCH`(41)와 요청 방법(메서드)만 다릅니다
`data: WithdrawRequest`	DELETE 요청인데도 **본문**을 받습니다 (HTTP가 허용합니다). 비밀번호를 주소에 넣으면 서버 기록과 브라우저 기록에 남기 때문입니다
`response: Response`	FastAPI가 넣어 주는 응답 객체입니다. 여기에 쿠키 삭제를 붙이면 204 응답에 함께 실려 갑니다

4. 흐름

토큰을 확인하고(06), 본문을 검사한 뒤, 46번(비밀번호 확인 → 탈퇴 상태, 사진 처리, 저장)을 실행합니다. 그다음 쿠키를 지우고 204로 응답합니다.

46번이 오류(400)를 던지면 쿠키 삭제 줄까지 오지 않습니다. 그래서 계속 로그인 상태로 남습니다.

5. 왜 이렇게

선택	이유
쿠키는 **탈퇴에 성공한 뒤에만** 지움	비밀번호를 틀렸는데 로그아웃되면 안 됩니다 (시험으로 확인했습니다)
쿠키를 지운 뒤에도 30번이 한 번 더 막음	다른 브라우저나 기기에 남은 쿠키는 이 응답으로 지울 수 없습니다. 그 쿠키로 재발급을 시도하면 30번 `get_active_user`가 `DELETED` 상태를 보고 401로 응답합니다

선택	이유
비밀번호는 본문으로 받음	주소(`?password=`)는 서버 접속 기록과 브라우저 방문 기록에 남습니다
204로 응답	탈퇴한 사람에게 돌려줄 정보가 없습니다

6. 시험 (`tests/test_user_routes.py`에 6개를 추가해서 모두 38개, 실제 앱과 MySQL, 임시 폴더로 실행)

시험	확인한 것
탈퇴 (실제 로그인 → 사진 올리기 → 재발급 200 → 탈퇴)	204, 본문 없음, 쿠키 삭제 헤더(`Max-Age=0`, `Path=/api/v1/auth`)를 확인했습니다. 그 뒤 남은 토큰 401, 재발급 401, 다시 로그인 401, 사진 404이고 DB는 `DELETED`입니다
비밀번호가 틀림	400 `WRONG_PASSWORD`이고, 쿠키 삭제 헤더가 없으며, 내 정보 200과 재발급 200이 그대로 됩니다
본문 없음, `{}`, 빈 문자열	422이고, DB는 `ACTIVE` 그대로입니다
토큰 없음	401

전체 시험 344개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 49 — `purge_withdrawn_accounts()`

항목	내용
파일	`studionx/backend/app/workers/retention.py`
작업	P1-09 비밀번호 변경·탈퇴: 탈퇴 3일 뒤 개인 정보 정리 (PRD 01 FR-108, 개인정보 처리방침 §4)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	48 `DELETE /users/me` · 50 `InquiryCreateRequest` (P1-12)

1. 하는 일

하루에 한 번 실행하는 **정리 작업**입니다. 탈퇴한 지 3일이 지난 계정에서 개인 정보를 비웁니다. 처리방침(privacy.html §4)에 적은 "탈퇴하면 3일 동안 보관한 뒤 비운다"는 약속을 실제로 지키는 곳입니다.

칸	정리 전	정리 후
`email`	`old.gone@example.com`	`deleted+85@withdrawn.invalid`
`nickname`	`old_gone`	`deleted_user_0000000085`
`password_hash`	`$2b$12$…`	`!` (어떤 비밀번호와도 맞지 않습니다)
`last_login_at`	마지막 로그인 시각	`None`
`profile_image_path`와 사진 파일	(남아 있다면)	`None`, 파일 삭제
`status`, `deleted_at`, 행 자체, 프로젝트		**그대로 둡니다**

계정	결과
탈퇴한 지 4일, 정확히 3일	정리합니다
탈퇴한 지 2일	그대로 둡니다. 3일 동안은 재가입을 막는 데 이메일과 닉네임을 쓰기 때문입니다
활동 중인 회원	그대로 둡니다
이미 모두 비운 계정	건너뜁니다 (세지도 않습니다)
다른 사람이 그 이메일로 가입하면서 이메일과 닉네임만 비켜 둔 계정	남아 있는 비밀번호 해시 등을 마저 비웁니다

돌려주는 값은 이번에 비운 계정의 수입니다.

2. 언제 쓰나

실행	언제
`python -m app.workers.retention`의 `main()`	하루에 한 번 실행합니다 (운영체제의 예약 작업이며, 배포할 때 정합니다). **`main()`은 P1-12에서** 문의 정리(`purge_old_inquiries`)와 함께 부르도록 그때 구현합니다

탈퇴 회원의 시간표

시점	상태	처리하는 곳
탈퇴하는 순간	`DELETED`로 바뀌고, 사진이 지워지며, 로그인과 재발급이 막힙니다	46, 48
3일까지	같은 이메일이나 닉네임으로 가입할 수 없습니다 (409 `REJOIN_TOO_SOON`)	P1-04
3일 뒤 누군가 그 값을 쓰려고 함	이메일과 닉네임만 바로 비켜 둡니다	P1-04 `release_withdrawn`
3일 뒤 첫 정리 작업	**나머지까지 모두 비웁니다**	**49**

추적 방지 (2026-10-07 결정 ②)

비켜 둔 값에는 회원 번호만 남고, 원래의 이메일, 닉네임, 비밀번호 해시, 마지막 로그인 시각, 사진은 남지 않습니다. 결정 ①(다른 사람에게 보이는 곳에는 'Deleted user'로 표시)과 결정 ③(회원 번호로 조회하는 공개 API를 두지 않음, P1-08 사진 주소 포함)과 함께 적용합니다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import or_, select
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.models import User
from app.models.user import UserStatus
from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn
```

`datetime` 줄과 `Session` 줄을 뺀 나머지는 모두 이번에 추가했습니다.

상수

```
# 다 비운 계정의 비밀번호 칸 값 — bcrypt 해시 모양이 아니라 verify_password 가 늘 False
# (칸이 NOT NULL 이라 빈칸 대신)
WIPED_PASSWORD_HASH = "!"
```

함수

```
def purge_withdrawn_accounts(db: Session, now: datetime) -> int:
    if now.tzinfo is not None:
        now = now.astimezone(UTC).replace(tzinfo=None)
    cutoff = now - timedelta(days=REJOIN_WAIT_DAYS)
    users = db.scalars(
        select(User).where(
            User.status == UserStatus.DELETED,
            User.deleted_at <= cutoff,
            or_(
                ~User.email.startswith("deleted+", autoescape=True),
                ~User.nickname.startswith("deleted_user_", autoescape=True),
```

```
                User.password_hash != WIPED_PASSWORD_HASH,
                User.last_login_at.is_not(None),
                User.profile_image_path.is_not(None),
            ),
        )
    ).all()

    photos = []
    for user in users:
        release_withdrawn(db, user)
        if user.profile_image_path:
            photos.append(user.profile_image_path)
        user.password_hash = WIPED_PASSWORD_HASH
        user.last_login_at = None
        user.profile_image_path = None
    db.commit()

    storage_dir = get_settings().storage_dir
    for relative in photos:
        (storage_dir / relative).unlink(missing_ok=True)
    return len(users)
```

문법 풀이

import 문법	뜻
`from sqlalchemy import or_, select`	`or_(A, B, …)`는 SQL의 `A OR B OR …`입니다
`from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn`	3일이라는 값과 비켜 두는 함수를 가입 쪽(P1-04)과 **같은 것으로** 씁니다

식	뜻
`now.astimezone(UTC).replace(tzinfo=None)`	한국 시각처럼 시간대가 붙은 값을 UTC로 바꾸고 시간대 표시를 뗍니다. DB는 UTC로 저장하기 때문입니다
`User.email.startswith("deleted+", autoescape=True)`	SQL `LIKE 'deleted+%'`가 됩니다. `autoescape`는 값 안의 `_`와 `%`를 글자 그대로 다루게 합니다 (`LIKE`에서 `_`는 "아무 글자 하나"를 뜻하기 때문입니다)
`~조건`	NOT입니다
`.is_not(None)`	`IS NOT NULL`입니다
`db.scalars(…).all()`	조건에 맞는 `User`들을 목록으로 가져옵니다

4. 흐름

1. 기준 시각을 지금으로부터 3일 전(UTC)으로 정합니다.
2. 탈퇴했고, 기준 시각보다 오래되었으며, **아직 남아 있는 정보가 하나라도 있는** 계정을 찾습니다.
3. 계정마다 이메일과 닉네임을 비켜 두고, 사진 경로를 기억해 둔 뒤, 비밀번호, 마지막 로그인 시각, 사진 경로를 비웁니다.
4. 한 번에 저장합니다.
5. 저장한 뒤 사진 파일을 지웁니다 (이미 없으면 넘어갑니다).
6. 비운 계정 수를 돌려줍니다.

5. 왜 이렇게

선택	이유
"남아 있는 정보가 하나라도 있는" 행만 고름	매일 실행해도 같은 행을 다시 세지 않습니다. 또 가입 때 이메일과 닉네임만 비켜 둔 행도 빠뜨리지 않습니다. 템플릿은 "이메일이 `deleted+`로 시작하지 않는 행"만 골랐는데, 그러면 그런 행의 비밀번호 해시가 영원히 남게 됩니다
`password_hash = "!"`	이 칸은 `NOT NULL`이라 비울 수 없습니다. `"!"`는 bcrypt 해시 모양이 아니라서 어떤 비밀번호로도 로그인할 수 없습니다 (44번 시험에서 확인했습니다)
`release_withdrawn`을 그대로 씀	비켜 두는 값이 가입 쪽과 항상 같아집니다. 여러 번 불려도 결과가 같습니다
`deleted_at`과 행을 남김	프로젝트(owner_id)가 이 행을 가리키고 있습니다. 탈퇴 시각도 기록으로 필요합니다
사진 파일은 저장한 **뒤에** 지움	38, 39번과 같은 순서입니다. 저장이 실패하면 파일도 남습니다
`now`를 인자로 받음 (함수 안에서 시각을 재지 않음)	시험에서 시각을 정해서 넣을 수 있습니다 ("정확히 3일" 같은 경계값)
시간대가 붙은 `now`도 받음	실행하는 쪽이 한국 시각을 넘겨도 틀리지 않게 합니다

알아 둘 점 (색인): `status`와 `deleted_at` 조건으로 회원 표를 훑습니다. 하루에 한 번만 실행하므로 회원이 많아도 부담이 적습니다.

6. 시험 (`tests/test_retention.py` 새 파일, 8개, 실제 MySQL과 임시 폴더로 실행)

시험	확인한 것
탈퇴한 지 4일 (사진, 로그인 기록, 프로젝트 있음)	1을 돌려주고, 이메일과 닉네임이 비켜지며, 해시는 `!`, 마지막 로그인과 사진 경로는 `None`이 되고 파일이 지워집니다. 상태, 탈퇴 시각, 프로젝트는 그대로입니다
탈퇴한 지 2일, 활동 중인 회원	0을 돌려주고 그대로 둡니다
정확히 3일	정리합니다
두 번 실행	처음에는 1, 두 번째에는 0입니다
가입 때 이메일과 닉네임만 비켜 둔 행	1을 돌려주고, 해시와 로그인 기록도 비웁니다
사진 파일이 이미 없음	오류 없이 정리합니다
한국 시각 21:00 (= UTC 12:00)으로 실행	정확히 3일로 계산해서 정리합니다
정리한 뒤	옛 비밀번호로 로그인하면 401이고, 옛 이메일과 닉네임으로는 가입할 수 있습니다

전체 시험 352개가 통과했고, `ruff` 검사도 통과했습니다.